

INFORME TÉCNICO MAESTRO — PaseCultural

Documento de estado real del proyecto, basado exclusivamente en el código presente en el repositorio al momento de escribirlo. No es un documento de planificación ni de opinión: describe lo que existe, cómo funciona y qué falta, tal como está implementado hoy.

1. RESUMEN GENERAL

Objetivo de la aplicación. PaseCultural es una plataforma de venta y administración de entradas para eventos culturales (teatro, música, centros culturales, productoras independientes). Cubre todo el ciclo: un organizador crea y publica un evento con sus funciones y tipos de entrada, el público compra sin necesidad de crear una cuenta, cada entrada se emite como un ticket individual con QR cifrado, el ingreso se controla en la puerta con una app de "Scanner" operada por personal sin cuenta de usuario tradicional, y el organizador (o el equipo de PaseCultural, rol "Developer") administra todo desde paneles propios.

Arquitectura general. Monorepo con dos carpetas independientes y sin capa intermedia: `backend` (API REST) y `frontend` (SPA), que se comunican directo por HTTP/JSON. No hay SSR, no hay BFF, no hay microservicios: es una API monolítica y un cliente SPA.

Tecnologías utilizadas — Frontend: React 19, Vite 8, React Router 7, Tailwind CSS 4, Clerk (`@clerk/clerk-react` + `@clerk/localizations` para español), Google Maps JS API (`@googlemaps/js-api-loader`), `qrcode.react` (render de QR), `qr-scanner` (lectura de QR por cámara, módulo Scanner), `jspdf` (generación de PDF de entradas en el navegador), `lucide-react` (iconografía). Sin librería de manejo de estado global (ni Redux ni Zustand ni similar — todo con Context de React + `useState/useEffect`), sin librería de formularios, sin librería de componentes UI de terceros (todo el sistema de diseño es propio, sobre Tailwind), sin `html2canvas`, sin i18n propio (textos en español hardcodeados; sólo la UI de Clerk está localizada), sin SDK de analítica/monitoreo, sin infraestructura de testing (no hay Jest/Vitest/Testing Library instalado).

Tecnologías utilizadas — Backend: Node.js con Express 5 (ESM puro, `type: module`), Prisma 6 como ORM sobre PostgreSQL, `@clerk/express` para autenticación de usuarios con cuenta, `jsonwebtoken` (JWT propio, sólo para sesiones de Scanner), `bcrypt` (no usado para contraseñas de usuario — Clerk las maneja— sino como parte de la infraestructura de hashing del módulo Scanner/verificación), `resend` (envío de email transaccional), `cloudinary` (imágenes), `pdfkit` (PDF de entradas del lado servidor, para el link público de recuperación), `qrcode` (generación de imágenes QR para los emails), `multer` (upload de archivos), `pg` (driver Postgres). Sin framework de testing, sin linter configurado, sin TypeScript.

Base de datos. PostgreSQL, gestionada enteramente por Prisma (schema versionado + migraciones SQL generadas). 35 migraciones aplicadas a la fecha de esta actualización (26 al momento de la redacción original de este informe, 2026-08-08; las 9 restantes son del subsistema WhatsApp — ver "Actualización al 13 de agosto de 2026").

Servicios externos integrados: Clerk (autenticación y gestión de cuentas de compradores registrados, organizadores y developer), Resend (todos los emails transaccionales), Cloudinary (imágenes de portada de evento y logo de organización), Google Maps JavaScript API (autocompletar direcciones, mapas de ubicación), **WhatsApp Business API / Meta Graph API** (webhook + envío de mensajes del bot conversacional para organizadores — integrado entre el 2026-08-09 y el 2026-08-12, posterior a la redacción original de este informe; ver "Actualización al 13 de agosto de 2026"). **Mercado Pago NO está integrado** — ver secciones 7 y 14, sigue siendo la ausencia más importante del proyecto.

2. MÓDULOS

Para cada módulo: objetivo, estado, % aproximado, funcionalidades implementadas y pendientes.

Autenticación

Objetivo: identificar quién es cada usuario y qué puede hacer. **Estado: completo para su alcance actual. ~95%.** Implementado: login/registro delegado 100% a los componentes prebuilt de Clerk; sincronización on-demand de la sesión de Clerk a una fila `User` propia (`POST /api/auth/sync`, sin webhook de Clerk); "adopción" automática de compras de invitado previas cuando alguien se registra con el mismo email que usó para comprar

sin cuenta; asignación de rol `DEVELOPER` por lista hardcodeda de emails en el código; promoción automática a `ORGANIZER` al crear una organización; roles `DEVELOPER/ORGANIZER/SCANNER/CUSTOMER`; suspensión de cuenta por Developer. El módulo Scanner tiene su **propio** sistema de autenticación, completamente separado de Clerk (JWT propio, ver módulo Scanners). Pendiente: no hay webhook de Clerk (`user.created`, etc.) — la sincronización depende de que el frontend llame a `/api/auth/sync` después de cada login; no hay flujo de "olvidé mi contraseña" propio (lo maneja Clerk); el rol `SCANNER` del enum `Role` de `User` está prácticamente en desuso (los scanners reales son filas `EventScanner`, no `User`).

Organizaciones

Objetivo: que una entidad (teatro, productora, etc.) pueda operar como organizador. **Estado: completo. ~90%.** Implementado: alta de organización (una por usuario, auto-promueve a `ORGANIZER`), edición de datos propios, baja propia, listado/detalle/aprobación/rechazo/suspensión por Developer, bloqueo de publicación de eventos si la organización no está `APPROVED`. Pendiente: no hay verificación de identidad más allá de los campos de texto que carga el propio organizador (CUIT, responsable, etc. — nada se valida contra un padrón externo); no hay onboarding de datos de cobro (ver Mercado Pago).

Eventos

Objetivo: que un organizador arme y publique un evento con sus funciones y entradas. **Estado: completo, con TRES caminos de creación/edición en paralelo (dos web + WhatsApp). ~90%.** Implementado: CRUD completo (crear/listar/editar/eliminar/publicar/despublicar/cancelar), motor conversacional tipo "chat" para crear eventos paso a paso (con posibilidad de retomar, editar cualquier sección desde una vista previa, y guardar como borrador), wizard clásico de formulario para editar eventos ya creados, programación de funciones (fecha única, rango, días de la semana, recurrencia), catálogo de tipos de entrada con overrides por función, enlaces del evento (redes/video, con detección automática de plataforma), ubicación con Google Maps, listado público con filtros (categoría, texto, fecha, gratis/pago, orden), detalle público por slug. Reglas de publicación (requiere organización aprobada, ubicación completa, al menos una función, al menos un tipo de entrada con precio y stock, cada función con al menos una asignación de entradas habilitada). **[Actualizado 2026-08-13]** El canal WhatsApp del motor conversacional (`ConversationChannel.WHATSAPP`), que este informe describía como "declarado pero sin adaptador", tiene desde el 2026-08-09 un adaptador completo y en uso (webhook de Meta, identificación por teléfono, selector de organización, y el mismo `EventCreationEngine` que usa la Web) — ver detalle completo, estado punto por punto y hallazgos de UX pendientes en "Actualización al 13 de agosto de 2026". Pendiente: no hay reordenamiento manual de eventos en el listado del organizador más allá del orden por fecha de creación.

Tipos de entradas

Objetivo: catálogo de tipos de entrada (General, VIP, etc.) por evento, con precio/stock/visibilidad, y sus asignaciones por función. **Estado: funcional, con una pantalla de sólo lectura desactualizada. ~80%.** Implementado: alta/edición/baja de tipos de entrada, overrides de precio/stock/visibilidad por función, catálogo dentro del wizard/chat de evento. Pendiente: la pantalla dedicada del organizador ("Tipos de entrada", ex-"Entradas") muestra la columna "Vendidas" **hardcodeada en 0** para toda fila, sin conectar a ningún dato real — no refleja ventas reales.

Cortesías

Objetivo: que el organizador emita entradas sin costo (sponsors, prensa, invitados, staff, artistas, familiares) sin mezclarlas con la venta real. **Estado: completo. ~90%.** Implementado: módulo propio dentro del panel de organizador (`/organizador/cortesias`), separado de "Entradas"/"Ventas"; asistente de 6 pasos (evento → función → tipo de entrada del catálogo existente, nunca uno nuevo → cantidad → motivo opcional [Sponsor/Prensa/Invitado/Staff/Artista/Familiar/Protocolo/Otro] + nota libre → entrega por "Compartir" o "Enviar por correo"); "Compartir" reutiliza exactamente la misma infraestructura de invitación del Scanner (Web Share API con fallback a copiar link/WhatsApp/QR); "Enviar por correo" dispara el mismo email de confirmación que recibe un comprador real, sin ningún template paralelo; la emisión reutiliza el mismo flujo de venta confirmada de punta a punta (mismo `Ticket`, mismo QR cifrado, mismo PDF, mismo Scanner) — la única diferencia funcional es `origin = COURTESY` en vez de `SALE` y precio 0; historial con filtros (evento/estado/motivo), estado derivado en vivo (nunca almacenado) y acciones por fila (compartir de nuevo, reenviar por correo, descargar PDF, copiar link, cancelar); estadísticas de cortesías (emitidas/utilizadas/pendientes/canceladas) integradas en "Estado de Funciones"; auditoría completa por emisión (quién, cuándo, evento, función, motivo, cantidad, email de destino si aplica, método de entrega). Restringido a roles `ORGANIZER / DEVELOPER`. Pendiente: un `DEVELOPER` no puede cancelar una cortesía de una organización que no es la propia (limitación heredada de `ticketAdmin.service.js`, no

ampliada a propósito); el historial no tiene todavía un filtro visible por canal de emisión, aunque la estructura de datos (`Ticket.origin`) ya lo permite sin cambios de arquitectura.

Compra

Objetivo: que cualquier persona pueda comprar entradas sin necesidad de crear una cuenta. **Estado: completo como flujo de checkout, sin pasarela de pago real. ~75%** (100% del flujo de checkout invitado, 0% de integración de pago real). Implementado: selección de función, selección de cantidad por tipo de entrada, resumen, datos del comprador (nombre/apellido/email/DNI, sin cuenta), confirmación "instantánea" (ver Mercado Pago — hoy no hay paso de pago real intermedio), recuperación de una compra en curso por `saleToken` en la URL (sobrevive a recargas de página), reintento con backoff/timeout si la confirmación tarda. Pendiente: **no existe ningún paso de pago real** — comprar hoy es funcionalmente gratis para cualquier evento, pago o no, del lado técnico (ver sección 7).

Confirmación / Mercado Pago

Objetivo: cobrar de verdad y confirmar la venta según el resultado del pago. **Estado: NO implementado. ~0-5%** (sólo el enum y el punto de extensión están preparados). Implementado: nada de integración real. Existe el valor `MERCADO_PAGO` en el enum `PaymentMethod` (nunca usado en la práctica, todas las ventas se crean con `MANUAL`), un campo `Sale.confirmedBy` nullable (documentado como preparado para que una confirmación automática por webhook no tenga un organizador asociado), y un único punto de extensión aislado en el frontend (`processPayment()` en `lib/payment/paymentGateway.js`) documentado explícitamente como "la única función que va a cambiar el día que se integre Mercado Pago". Hoy esa función crea la venta y la confirma en el mismo paso, sin pasarela ni redirección a ningún checkout externo. Confirmación manual por organizador (`POST /api/sales/:id/confirm`) sí existe y es el único mecanismo real de confirmación (pensado para pagos en efectivo/transferencia cargados a mano). Pendiente: **todo** — SDK, Checkout Pro o Checkout API, webhook de notificación, conciliación de estados, manejo de pagos rechazados/pendientes, onboarding de cuenta de cobro del organizador (la pantalla de Configuración ya tiene el placeholder visual para esto, sin ninguna lógica detrás).

Recuperación de entradas

Objetivo: que alguien que compró sin cuenta pueda volver a ver/descargar sus entradas. **Estado: completo. ~95%.** Implementado: búsqueda por email+DNI (sin revelar si existe o no una compra), código de verificación de 6 dígitos por email como segundo factor (con hash, expiración, límite de intentos, cooldown de reenvío y rate limit por IP), pantalla intermedia "Compra encontrada" antes de exponer cualquier dato, reenvío del email de confirmación completo, descarga del PDF completo de la compra, reutilización total del flujo de compra existente (`PurchaseWizard / SuccessStep`) vía `saleToken` para "Ver mis entradas". Pendiente: nada identificado como incompleto dentro de su alcance.

Correos

Objetivo: todas las notificaciones transaccionales por email. **Estado: completo para lo que existe, acotado en alcance. ~85%** (de lo que el proyecto necesita hoy; ver pendientes). Implementado: 3 emails reales (confirmación de compra con QRs adjuntos + PDF, código de verificación de Scanner, código de verificación de recuperación de compra) — detalle completo en sección 8. Pendiente: no hay email de bienvenida, no hay email de aprobación/rechazo de organización (el organizador sólo se entera si vuelve a mirar el panel), no hay notificación al organizador cuando se registra una venta nueva.

Entradas (administración)

Objetivo: que el organizador pueda administrar entradas ya vendidas (cancelar, rehabilitar, marcar usada, eliminar, ver historial). **Estado: completo, backend y frontend. ~90%.** Implementado: modelo de datos con auditoría completa (`TicketAuditLog` , append-only) y check-ins históricos (`CheckIn` , ya no 1:1 con la entrada — una entrada reactivada puede volver a tener ingresos), 5 operaciones administrativas individuales (cancelar/rehabilitar/reactivar/marcar usada manualmente/eliminar con soft delete) cada una con su fila de auditoría, acciones masivas sobre selección múltiple (cancelar/rehabilitar/eliminar), pantalla del organizador con selector de evento/función, buscador server-side (número/nombre/email/DNI), tarjetas de resumen por estado, selección múltiple con checkbox, drawer lateral de detalle con check-ins y auditoría completos. Pendiente: la acción masiva "Exportar seleccionadas" está presente en la UI pero es un stub (muestra un toast "se preparará en una próxima iteración", no genera ningún archivo); los filtros por estado individuales (pills "Disponibles/Utilizadas/Canceladas/Reintegradas/Eliminadas") quedaron sin punto de entrada en la UI actual — el

estado que los controla (`statusFilter`) sigue existiendo y viajando al backend, pero no hay ningún control visible que lo cambie (ver sección 14, deuda técnica).

Scanners

Objetivo: operar el control de acceso en la puerta de un evento, sin necesidad de una cuenta de usuario tradicional. **Estado: completo. ~95%.** Implementado: alta de invitaciones por el organizador (una o varias, por puerta), registro público único por invitación (nombre/apellido/DNI/email/teléfono + código de 6 dígitos), portal de acceso recurrente "Soy Scanner" (email + código de 6 dígitos, sin contraseña, sin Clerk) para todo ingreso posterior a la invitación, sesión propia por JWT firmado (24hs), verificación de que el scanner sigue `ACTIVE` en cada request (una desactivación del organizador corta el acceso al instante), dashboard previo (identidad, evento, puerta, estado, último acceso, entradas validadas hoy), selección de evento/función, lector de QR con cámara, historial de escaneos, estadísticas de función, validación con garantías de concurrencia verificadas (ver sección 7). Pendiente: el rol `SCANNER` de `User`/Clerk no se usa en este flujo (es intencional, no una carencia); no hay una vista de "todas las puertas en vivo" para el organizador durante el evento (sólo historial/estadísticas por función).

Dashboard Organizador

Objetivo: panorama general del organizador. **Estado: completo, con datos reales de punta a punta. ~90%.** Implementado: banner de estado de la organización (pendiente/rechazada/suspendida/aprobada); selector de evento destacado (en curso/próximo/finalizado) que alimenta todo lo demás de la pantalla; hero del evento seleccionado con ocupación real; resumen del evento en 4 bloques — **Comercial** (recaudación, entradas vendidas, ticket promedio — sólo `origin=SALE`, nunca incluye cortesías), **Emisión** (entradas emitidas totales + desglose por canal con nombres amigables, ej. "□ Vendidas" / "□ Cortesías"), **Accesos** (ingresadas, pendientes de ingreso, canceladas si aplica) y **Ocupación** (capacidad, emitidas, disponibles, % de ocupación) — todos calculados 100% en el backend (`functionCapacity.service.js`), nunca recalculados en el cliente; actividad reciente del evento (ventas, check-ins, acciones de auditoría, altas de scanner) combinada en una sola línea de tiempo; grilla "Estado de mis eventos" (todos los eventos de la organización) con capacidad/ocupación reales vía un endpoint batcheado propio (`GET /api/events/mine/stats`); tabla de últimas ventas del evento seleccionado. "Capacidad total"/"Entradas emitidas" del bloque Ocupación suman todas las funciones vigentes del evento a propósito (capacidad real de toda la temporada, no de una función suelta) — verificado correcto en una auditoría de código punta a punta. Pendiente: la "recaudación" que se muestra es la suma de ventas ya `CONFIRMED` en la base — sigue sin existir un cobro real (ver Mercado Pago).

Dashboard Developer

Objetivo: panorama general de la plataforma para el equipo de PaseCultural. **Estado: maqueta visual, sin ningún dato real. ~10%** (sólo el layout/diseño). Implementado: layout completo con tarjetas de estadísticas, gráfico de ventas de 7 días, gráfico de dona de estado de entradas, lista de próximos eventos, actividad reciente. Pendiente: **absolutamente todo el contenido es un valor hardcodedo en el código** (usuarios, eventos, entradas vendidas, ventas, la serie del gráfico, los eventos "próximos" con fechas de 2025, la actividad reciente) — no hay ninguna llamada a la API en todo el archivo. Los botones "Ver todos"/"Ver todas" no tienen acción.

Dashboard Scanner

Objetivo: pantalla previa al lector de QR para el operador de puerta. **Estado: completo. ~95%.** (Ya cubierto en el módulo "Scanners" — se lista aparte porque el pedido lo pide como ítem propio.)

Configuración

Objetivo: que el organizador administre los datos de su organización. **Estado: completo salvo cobros. ~70%.** Implementado: edición de todos los datos de la organización (nombre, tipo, CUIT, contacto, ubicación, redes, logo, descripción). Pendiente: la tarjeta "Datos bancarios / Mercado Pago" es un placeholder estático ("Próximamente"), sin ningún campo ni lógica — coincide con que Mercado Pago no está integrado.

Mercado Pago

Ver arriba ("Confirmación / Mercado Pago") — se repite acá porque el pedido lo pide como módulo propio. **Estado: no implementado. ~0-5%.**

Medios (Cloudinary)

Objetivo: subir y administrar imágenes (portadas de evento, logos de organización). **Estado: completo. ~95%.**
Implementado: subida genérica de imágenes (5MB máx, PNG/JPEG/WEBP), borrado por `publicId`, usado tanto para portadas de evento como logos de organización. Sin pendientes identificados dentro de su alcance.

3. FRONTEND

Todas las pantallas, agrupadas por área. Para cada una: ruta, propósito, estado, componentes principales.

Públicas (sin autenticación, dentro de `PublicShell`)

Ruta	Propósito	Estado	Componentes principales
<code>/</code>	Home / marketplace	Terminada	<code>HeroCarousel</code> , <code>CategoryFilterBar</code> , <code>EventsCarousel</code> (x3), <code>TrustBar</code> , <code>RecoverPurchaseSection</code> , <code>ScannerPortalSection</code>
<code>/eventos</code>	Listado con filtros/búsqueda/orden	Terminada	<code>EventCard</code> , filtros por <code>queryString</code>
<code>/evento/:slug</code>	Detalle público de evento	Terminada	<code>MediaEmbed</code> , <code>SocialLinks</code> , <code>LocationMap</code>
<code>/comprar</code>	Wizard de compra (nuevo o recuperado por <code>saleToken</code>)	Terminada como checkout; sin pago real	<code>PurchaseWizard</code> + 5 steps, <code>PurchaseOverlay</code>
<code>/mis-entradas</code>	Entradas del usuario con cuenta	Terminada	<code>TicketCard</code> , <code>TicketDetailModal</code> , <code>TicketQrModal</code> / <code>TicketQrFullscreen</code>
<code>/recuperar-compra</code>	Recuperación de compra sin cuenta	Terminada	flujo propio de código de 6 dígitos, pantalla "Compra encontrada"
<code>/scanner/invitacion/:token</code>	Registro único de scanner por invitación	Terminada	<code>ScannerInvitationClaim</code>
<code>/scanner/portal</code>	Login recurrente de scanner (email+código)	Terminada	<code>ScannerPortal</code>
<code>/para-organizadores</code>	Landing comercial	Terminada (contenido estático)	<code>Hero</code> / <code>Features</code> / <code>HowItWorks</code> / <code>CtaBanner</code> / <code>TrustBar</code> propios de la landing
<code>/como-funciona</code>	Explicación + FAQ	Terminada (contenido estático)	<code>FaqAccordion</code> , <code>SecurityCard</code> , <code>StepCard</code>
<code>/perfil</code>	Perfil del usuario	Stub (<code>ComingSoon</code>)	—
<code>/iniciar-sesion</code>	Login	Terminada (componente prebuilt de Clerk)	<code><SignIn/></code> de Clerk
<code>/registro</code>	Registro	Terminada (componente prebuilt de Clerk)	<code><SignUp/></code> de Clerk

Scanner (shell propio, sin Clerk)

Ruta	Propósito	Estado
<code>/scanner</code>	Toda la app de Scanner (dashboard → selección → lector)	Terminada

Auth-gated fuera del panel

Ruta	Propósito	Estado
<code>/bienvenida</code>	Redirección post-login según rol	Terminada

Ruta	Propósito	Estado
/organizador/nueva-organizacion	Alta de organización	Terminada

Panel Developer (RoleGuard["deveLoper"])

Ruta	Propósito	Estado
/developer	Dashboard general	Maqueta, sin datos reales
/developer/organizaciones	Aprobar/rechazar/suspender organizaciones	Terminada
/developer/usuarios	Gestión de usuarios (rol, estado, baja)	Terminada

Panel Organizador (RoleGuard["organizer"] , bajo /organizador)

Ruta	Propósito	Estado
/organizador	Dashboard	Terminada, con datos reales (ver §9)
/organizador/eventos	Listado/CRUD de eventos	Terminada
/organizador/eventos/nuevo	Creación conversacional de evento	Terminada
/organizador/eventos/:id/editar	Edición por wizard clásico	Terminada
/organizador/entradas	Administración de entradas vendidas	Terminada, con deuda técnica puntual (ver §14)
/organizador/tipos-de-entrada	Catálogo de tipos de entrada (solo lectura)	Terminada, con columna "Vendidas" siempre en 0
/organizador/cortesias	Landing del módulo Cortesías	Terminada
/organizador/cortesias/emitir	Asistente de emisión de cortesía	Terminada
/organizador/cortesias/historial	Historial de cortesías emitidas	Terminada
/organizador/ventas	Ventas/órdenes	Stub permanente
/organizador/scanners	Gestión de scanners de un evento	Terminada
/organizador/scanners/nuevo	Alta conversacional de invitaciones de scanner	Terminada
/organizador/configuracion	Datos de la organización	Terminada salvo cobros (placeholder)

Ruta comodín * dentro del panel → pantalla local "Página no encontrada".

4. BACKEND

Servicios (backend/src/services/): auth.service.js , user.service.js , organization.service.js , event.service.js , functionCapacity.service.js (única fuente de capacidad/emitidas/vendidas/ingresadas/canceladas — por función, por evento y batcheada para todos los eventos del organizador), sale.service.js , courtesy.service.js (emisión/historial/estadísticas/cancelación de cortesías, reutilizando sale.service.js / ticketAdmin.service.js en vez de duplicar lógica de venta), saleRecoveryVerification.service.js , ticket.service.js , ticketAdmin.service.js , scanner.service.js , scannerInvitation.service.js , scannerLogin.service.js , eventScanner.service.js , scannerRead.service.js , media.service.js , más el subárbol email/ (sendSaleConfirmationEmail.service.js , sendScannerVerificationCode.service.js , sendSaleRecoveryVerificationCode.service.js , ticketQrImages.js , ticketsPdf.js , formatDateAR.js) y el subárbol conversation/ (EventCreationEngine.js , EventServicePort.js , steps/ , inputHandlers/ , errorMessages.js). **[Agregado 2026-08-13, subsistema WhatsApp, no existía en la redacción original]** whatsapp.service.js (envío/parseo Meta), whatsappOrganizerBot.service.js (textos, parseo de fechas/horarios compactos, formateo de respuestas), whatsappOrganizerDiscovery.service.js (resolución de organización por teléfono), whatsappOrganizerLink.service.js , whatsappPendingStepInput.service.js , whatsappOrganizationLocation.service.js , whatsappMediaUpload.service.js — detalle de cada uno en "Actualización al 13 de agosto de 2026".

Controllers (backend/src/controllers/): un archivo por dominio, espejo casi 1:1 de los servicios: auth.controller.js , user.controller.js , organization.controller.js , event.controller.js , eventScanner.controller.js , functionCapacity.controller.js , sale.controller.js , courtesy.controller.js ,

`ticket.controller.js`, `ticketAdmin.controller.js`, `scanner.controller.js`, `scannerRead.controller.js`, `scannerInvitation.controller.js`, `scannerAuth.controller.js`, `media.controller.js`, y (agregado 2026-08-13) `whatsapp.controller.js` (webhook + todo el árbol de sub-flujos conversacionales, es el controller más grande del proyecto — ver actualización). Todos siguen el mismo patrón: sólo validan `req`, llaman al service y devuelven la respuesta; la lógica de negocio vive exclusivamente en los services (`whatsapp.controller.js` es la única excepción parcial: por diseño concentra los sub-flujos de interpretación de texto libre, ver actualización).

Routes (backend/src/routes/): `auth.routes.js`, `user.routes.js`, `organization.routes.js`, `media.routes.js`, `event.routes.js` (incluye las sub-rutas de scanners, capacidad/estadísticas y administración de entradas de un evento), `conversation.routes.js`, `sale.routes.js`, `courtesy.routes.js`, `ticket.routes.js`, `scanner.routes.js`, `scannerInvitation.routes.js`, `scannerAuth.routes.js`, y (agregado 2026-08-13) `whatsapp.routes.js` (GET/POST /api/whatsapp/webhook, público — Meta no manda ningún header de sesión). Ver inventario completo de endpoints en la sección 10.

Middlewares (backend/src/middlewares/): exactamente 4 archivos — `requireAuth.js` (sólo confirma que hay sesión de Clerk), `requireRole.js` (variádico, resuelve y adjunta el `User` local, 401/403 según corresponda), `requireScannerSession.js` (JWT propio del módulo Scanner, revalida `ACTIVE` contra la base en cada request), `rateLimit.js` (limitador en memoria por IP, sin dependencia externa). CORS y el manejo global de errores no son middlewares propios sino configuración inline en `app.js` (`cors()`) y el módulo `errors/errorHandler.js`.

Jobs. No existe ningún sistema de jobs/colas/tareas programadas en el proyecto (sin cron, sin worker, sin cola de mensajes). Todo procesamiento es síncrono dentro del ciclo request/response.

Emails. Ver sección 8 completa.

Utilidades (backend/src/utils/): entre otras, `getUserByClerkId.js`, `validateEmail.js`, `validateBuyerDocument.js` (normalización/validación de DNI, compartida por compra, recuperación y auditoría de entradas), `validateOrganization.js`, `organizationTrust.js` (`canPublishEvents`), `generateSlug.js`, `mediaParser.js` (detección de plataforma de un link pegado), `verificationCode.js` (generación/hash/comparación de códigos de 6 dígitos, compartida entre Scanner y recuperación de compra), `withTimeout.js` (compartido por los tres servicios de email), `htmlEscape.js`, `ticketNumber.js`, `calendarDate.js` (única infraestructura del backend para fechas de calendario "YYYY-MM-DD" — parseo/normalización/combinación con hora en la timezone oficial de la plataforma, -03:00 fijo, nunca `new Date(string)` directo; usada por todo el motor conversacional de creación de eventos).

Configuración (backend/src/config/): `prisma.js` (cliente singleton), `resend.js` (cliente Resend + config de remitente, validación perezosa), `cloudinary.js`, `qrEncryption.js` (cifrado AES-256-GCM del secreto de cada QR), `scannerSession.js` (firma/verificación del JWT propio de Scanner).

5. BASE DE DATOS

Todas las entidades del `schema.prisma` (**22 modelos, 22 enums a la fecha de esta actualización — 18/21 al momento de la redacción original; los 4 modelos nuevos son del subsistema WhatsApp: `WhatsappLinkChallenge`, `WhatsappOrganizerLink`, `WhatsappPendingOrganizationSelection`, `WhatsappPendingStepInput`, más el campo `WhatsappPendingStepInput` -relacionado agregado a `ConversationState`; detalle de propósito de cada uno en "Actualización al 13 de agosto de 2026"**), con propósito, relaciones y uso actual.

- **User** — cuenta de cualquier persona (comprador, organizador, developer); `clerkId` nullable para soportar compra de invitado. Relaciona con `Organization` (dueño), `Sale` (comprador), `Ticket` (comprador/dueño de la entrada, roles separados a propósito para una futura transferencia de entradas). Uso: activo, es el centro de identidad de toda la app salvo el módulo Scanner.
- **Organization** — una entidad organizadora, dueña de sus eventos. Relaciona 1 a 1 con su `owner User`, 1 a N con `Event`. Uso: activo, con ciclo de aprobación por Developer.
- **Event** — un evento cultural. Relaciona con `Organization`, y 1 a N con `EventFunction`, `TicketType`, `EventLink`, `Sale`, `Ticket`, `EventScanner`. Uso: activo, es la entidad central del catálogo.
- **EventLink** — un link asociado a un evento (red social, video promocional, etc.), con metadata de embed resuelta del lado servidor. Uso: activo.
- **EventFunction** — una fecha/función concreta de un evento (nunca el precio/stock, que vive en `TicketType`). Relaciona con `FunctionTicketType`, `Sale`, `Ticket`. Uso: activo.
- **TicketType** — catálogo reusable de tipos de entrada de un evento (precio, stock, nombre). Relaciona con `FunctionTicketType`, `SaleItem`, `Ticket`. Uso: activo.

- **FunctionTicketType** — asigna un `TicketType` a una `EventFunction` concreta, con overrides opcionales de precio/stock/visibilidad. Uso: activo.
- **ConversationState** — estado persistido del motor conversacional de creación de eventos (borrador en curso, paso actual, historial de navegación). Uso: activo mientras dura la conversación; se vuelve irrelevante una vez que el evento se publica/guarda.
- **Sale** — una venta/orden de compra. Relaciona con `User` (comprador), `Event`, `EventFunction`, y 1 a N con `SaleItem / Ticket`. Incluye todo el seguimiento de envío del email de confirmación. Tiene `origin` (enum `SaleOrigin`: `SALE / COURTESY`, default `SALE`) — distingue una venta real de una cortesía sin duplicar el modelo; toda métrica comercial (recaudación, Mercado Pago cuando exista) filtra explícitamente por `origin=SALE`. Uso: activo, es la entidad central de la compra.
- **CourtesyIssuance** — metadata de una cortesía (motivo, nota libre, método de entrega, quién la emitió), 1 a 1 con la `Sale` que la representa — nunca un sistema de tickets paralelo, la cortesía usa exactamente el mismo `Sale / Ticket / TicketQr` que una venta real, sólo con `origin=COURTESY` y precio 0. Uso: activo.
- **SaleRecoveryVerification** — sesión de código de 6 dígitos para el flujo de recuperación de compra, con clave única por par email+DNI normalizado (no por venta, porque un mismo par puede matchear varias). Uso: activo, vida corta (se invalida tras usarse).
- **SaleItem** — línea de detalle de una venta (tipo de entrada + cantidad + precio congelado al momento de la compra). Uso: activo.
- **Ticket** — una entrada individual y escaneable, generada recién al confirmarse la venta. Relaciona con `Sale`, `Event`, `EventFunction`, `TicketType`, `User` (comprador y dueño), y 1 a N con `TicketQr`, `CheckIn`, `ScanAttempt`, `TicketAuditLog`. Tiene su propio `origin` (mismo enum `SaleOrigin`, copiado de `Sale.origin` al emitirse) — denormalizado a propósito porque Prisma no puede agrupar por un campo de una relación: es lo que permite que `functionCapacity.service.js` desglose emitidas/vendidas por canal (y cualquier canal futuro — `STAFF/PRESS/VIP/etc.` — sin tocar código, sólo agregando el valor al enum) sin un join costoso. El Scanner también lo usa para mostrar el canal de la entrada escaneada (□ venta / □ cortesía). Uso: activo, es la entidad central del control de acceso.
- **TicketQr** — el secreto del QR de una entrada, cifrado de forma reversible (nunca en texto plano, nunca como imagen persistida). Uso: activo.
- **CheckIn** — historial de ingresos de una entrada (ya no es 1 a 1 con `Ticket`: una entrada reactivada puede volver a tener check-ins). Guarda origen (escaneo real o carga manual), puerta, scanner y dispositivo. Uso: activo.
- **ScanAttempt** — auditoría de TODO intento de escaneo, válido o no (a diferencia de `CheckIn`, que sólo registra los válidos). Uso: activo, es el log completo para detectar QR reusados/compartidos.
- **TicketAuditLog** — bitácora append-only de toda acción administrativa sobre una entrada (cancelar/rehabilitar/reactivar/marcar usada/eliminar), con actor, motivo y transición de estado. Uso: activo, nunca se actualiza ni se borra por código.
- **EventScanner** — una persona/puesto de scanner para un evento puntual, sin cuenta de Clerk. Contiene tanto el ciclo de vida de la invitación como los datos de verificación por código. Uso: activo, es la identidad completa del módulo Scanner.

Enums: `Role`, `UserStatus`, `OrganizationStatus`, `OrganizationType`, `EventStatus`, `EventVisibility`, `FunctionStatus`, `ConversationChannel`, `ConversationStatus`, `SaleStatus`, `SaleOrigin` (`SALE / COURTESY` — el discriminador comercial/operativo de toda la plataforma), `PaymentMethod`, `EmailDeliveryStatus`, `TicketStatus`, `ScanResult`, `CourtesyReason` (`Sponsor/Prensa/Invitado/Staff/Artista/Familiar/Protocolo/Otro`), `CourtesyDeliveryMethod` (`SHARE / EMAIL`), `CheckInSource`, `TicketAuditAction`, `TicketAuditActorType`, `EventScannerStatus`.

6. FLUJOS IMPLEMENTADOS

Registro organizador. Un usuario con sesión de Clerk (cualquier rol previo) llama a crear una organización → si no tenía una, se crea en estado `PENDING` y su rol pasa de `CUSTOMER` a `ORGANIZER` automáticamente → puede operar el panel de organizador y armar eventos en borrador desde el primer momento → sólo puede **publicar** un evento una vez que un Developer aprueba la organización.

Creación de evento. Dos caminos posibles, ambos terminan en las mismas tablas: (a) motor conversacional paso a paso, que arma un `draftEvent` en JSON dentro de `ConversationState` y recién crea el `Event` real al confirmar "Publicar"/"Guardar borrador" en la vista previa; (b) wizard clásico de formulario, usado siempre para **editar** un evento ya existente (no para crear uno nuevo). Ambos pasan por las mismas validaciones de

publicación (organización aprobada, ubicación completa, al menos una función y un tipo de entrada con precio/stock).

Compra. El comprador elige función y cantidades sin necesidad de cuenta → completa nombre/apellido/email/DNI → se crea la `Sale` en estado `PENDING` → se confirma en el mismo paso (hoy no hay pasarela de pago real intermedia) → al confirmar se generan los `Ticket` individuales con su `TicketQr` cifrado → se dispara el email de confirmación con los QR y el PDF adjuntos.

Confirmación. Hoy existen dos caminos de confirmación: automática (la que dispara el propio flujo de compra, simulando un pago instantáneo) y manual (el organizador confirma una venta cargada a mano, por ejemplo un pago en efectivo). No existe un tercer camino por webhook de pasarela de pago porque no hay pasarela integrada.

Cortesías. El organizador elige evento → función (se saltea si sólo hay una) → tipo de entrada del catálogo existente → cantidad → motivo opcional → método de entrega → se crea la misma `Sale / Ticket / TicketQr` que generaría una venta real, pero con `origin=COURTESY` y precio 0, reutilizando el mismo `confirmSaleService` (con la única diferencia de que "Compartir" no dispara el email automático, para no mandarlo dos veces si el organizador también decide compartir el link). Queda auditada en `CourtesyIssuance` y nunca afecta ninguna métrica comercial (recaudación, ticket promedio, etc.), que siguen filtrando exclusivamente por `origin=SALE`.

Generación QR. Al confirmarse la venta, cada `Ticket` recibe un `TicketQr` cuyo secreto se genera y se cifra (AES-256-GCM) antes de guardarse — nunca se persiste en texto plano ni como imagen. El QR que ve el comprador se arma al vuelo (`ticketId.secretDescifrado`) cada vez que hace falta mostrarlo (email, "Mis entradas", PDF), nunca se guarda una imagen del QR en la base.

Correo. Disparado automáticamente tras la confirmación de una venta (por cualquiera de los dos caminos), con reintento idempotente (nunca duplica el envío si se llama más de una vez para la misma venta) y reenvío manual disponible tanto para el comprador (por token público) como para el organizador/developer (por id interno, autenticado).

Recuperación. El comprador sin cuenta busca por email+DNI (respuesta siempre genérica, no revela si existe o no una coincidencia) → si hay una compra real, se envía un código de 6 dígitos al correo → verificado el código, recién ahí se revela la compra (una pantalla intermedia "Compra encontrada", nunca los datos completos antes del código) → desde ahí puede ver sus entradas (reutilizando el flujo de compra por `saleToken`), reenviar el email o descargar el PDF completo.

Scanner. Un organizador genera invitaciones para una puerta → la persona invitada se registra una única vez (datos personales + código de 6 dígitos) → de ahí en más, todo acceso pasa por el Portal Scanner (email + código de 6 dígitos, sin contraseña) → sesión propia por JWT → dashboard previo → selección de evento/función → lector de cámara → cada escaneo válido marca la entrada como usada de forma atómica (a prueba de dos scanners escaneando el mismo QR al mismo tiempo) y registra un `CheckIn` + un `ScanAttempt`.

Administración de entradas. El organizador, desde el panel, puede cancelar, rehabilitar, reactivar (una entrada ya usada), marcar como usada manualmente (sin escaneo real) o eliminar (soft delete) cualquier entrada de sus eventos, individualmente o en lote — cada acción queda auditada.

Auditoría. Toda acción administrativa sobre una entrada (nunca un escaneo normal, que ya tiene su propio registro en `ScanAttempt / CheckIn`) genera una fila en `TicketAuditLog` con quién, cuándo, desde qué estado a cuál y por qué motivo (opcional) — nunca se actualiza ni se borra una fila de esa tabla por código.

7. SEGURIDAD

Autenticación. Dos sistemas completamente separados y deliberadamente no unificados: (1) Clerk para cualquier persona con cuenta (comprador registrado, organizador, developer) — el backend nunca valida contraseñas ni sesiones por su cuenta, delega 100% en el SDK de Clerk; (2) un sistema propio, sin Clerk, para el módulo Scanner — JWT firmado por el propio backend (`SCANNER_SESSION_SECRET`, 24hs de vigencia), emitido sólo tras verificar un código de 6 dígitos.

Autorización. Basada en rol (`Role` de `User`, resuelto siempre server-side vía `requireRole`, nunca confiado desde el cliente) para todo lo que no es del módulo Scanner. Dentro del panel de organizador, además del rol, cada acción revalida que el recurso (evento, scanner, entrada) pertenezca a la organización del que hace el pedido — nunca alcanza con tener rol `ORGANIZER`, tiene que ser dueño del recurso puntual.

Permisos. El rol `DEVELOPER` es el único que puede aprobar/rechazar organizaciones, cambiar el rol o estado de cualquier usuario, o listar todo el sistema sin acotarse a una organización propia. El rol `ORGANIZER` está siempre

acotado a los eventos de su propia organización.

Tokens. Tres tipos de token público distintos, cada uno con un propósito único y sin superposición: `Sale.publicRecoveryToken` (autoriza confirmar/consultar/recuperar una venta puntual sin cuenta), `EventScanner.invitationToken` (autoriza el registro único de un scanner — después de la activación, ya no sirve para volver a entrar, sólo dentro de una ventana de gracia de 2 minutos pensada exclusivamente para un doble-envío accidental del mismo formulario), y el JWT de sesión de Scanner (autoriza operar el lector, revalidado contra la base en cada request).

QR. El secreto de cada entrada se genera con un generador criptográfico seguro, se cifra de forma reversible antes de guardarse (la clave vive fuera de la base, en una variable de entorno) y nunca se persiste en texto plano ni como imagen. La validación de un escaneo compara el secreto provisto contra el descifrado en tiempo constante (protección contra timing attacks).

Protección contra reutilización. Verificada explícitamente con pruebas de concurrencia real (múltiples escaneos simultáneos del mismo QR, incluyendo hasta 20 en paralelo sobre la misma entrada): la garantía real es un `UPDATE` condicional atómico sobre el estado de la entrada dentro de una transacción — nunca puede haber dos escaneos válidos para la misma entrada, sin importar cuántos scanners la escaneen al mismo tiempo ni la velocidad del servidor.

Rate limits. Limitador propio en memoria (sin dependencia externa, no apto para múltiples instancias del servidor sin adaptarlo) aplicado a los endpoints públicos "adivinables": búsqueda de recuperación de compra, reenvío de código de recuperación, verificación de código de recuperación, registro/reenvío/verificación de invitación de scanner, login/reenvío/verificación del Portal Scanner.

Recuperación. Ver flujo completo en la sección 6 — diseñada explícitamente para no revelar nunca si un email/DNI existen en el sistema antes de verificar el código.

8. CORREOS

Tres emails implementados, los tres vía Resend:

1. **Confirmación de compra** — se dispara automáticamente al confirmarse una venta (por cualquiera de los dos caminos de confirmación) y también puede reenviarse manualmente (por el comprador vía token público, o por organizador/developer vía id interno). Contiene: los QR de cada entrada embebidos como imagen y un PDF adjunto con todas las entradas de la venta. Envío protegido contra duplicados con un reclamo atómico (nunca se manda dos veces para el mismo intento simultáneo).
2. **Código de verificación de Scanner** — se dispara tanto en el registro único por invitación como en cada login por el Portal Scanner. Contiene el código de 6 dígitos, vigente 10 minutos.
3. **Código de verificación de recuperación de compra** — se dispara al buscar una compra por email+DNI (sólo si hay una coincidencia real, aunque la respuesta al usuario sea siempre la misma exista o no). Contiene el código de 6 dígitos, vigente 10 minutos, sin mencionar ningún dato de la compra.

Qué falta. No hay email de bienvenida al crear una cuenta, no hay notificación al organizador cuando su organización es aprobada/rechazada (tiene que volver a mirar el panel), no hay notificación de venta nueva al organizador. Los emails transaccionales propios de Clerk (verificación de cuenta, magic link, etc.) los maneja Clerk por su cuenta, fuera de este código.

9. DASHBOARDS

Developer. Muestra tarjetas de estadísticas generales, un gráfico de ventas de 7 días, un gráfico de dona de estado de entradas, próximos eventos y actividad reciente — **absolutamente todo con datos hardcoded en el código**, sin ninguna llamada a la API. Falta: conectar cada sección a datos reales (no existe siquiera el intento de fetch).

Organizador. Banner de estado de la organización, selector de evento destacado, hero con ocupación real, resumen del evento en 4 bloques (Comercial/Emisión/Accesos/Ocupación — todos calculados en el backend, única fuente de verdad compartida con la pantalla de Entradas), actividad reciente, grilla "Estado de mis eventos" y últimas ventas — todo con datos reales, incluida la capacidad total sumada correctamente a través de todas las funciones vigentes del evento. Falta: la "recaudación" mostrada es la suma de ventas confirmadas en la base, no un cobro real (ligado a Mercado Pago).

Scanner. Completo — identidad del scanner, evento, puerta, estado, último acceso y entradas validadas hoy, todo con datos reales resueltos del lado servidor. Sin pendientes identificados.

10. APIs

Endpoints públicos (sin autenticación)

- GET /api/health
- GET /api/events/public, GET /api/events/public/:slug, GET /api/events/categories
- POST /api/sales, POST /api/sales/:token/confirm-by-buyer, GET /api/sales/:token/status, GET /api/sales/:token/pdf, POST /api/sales/:token/resent-email
- POST /api/sales/recover, POST /api/sales/recover/resent, POST /api/sales/recover/verify
- GET /api/scanner-invitations/:token, POST /api/scanner-invitations/:token/register, POST /api/scanner-invitations/:token/resent, POST /api/scanner-invitations/:token/verify
- POST /api/scanner-auth/request-code, POST /api/scanner-auth/resent-code, POST /api/scanner-auth/verify

Endpoints privados (requieren sesión de Clerk, o sesión propia de Scanner)

Auth: POST /api/auth/sync.

Usuarios (sólo Developer): GET /api/users, GET /api/users/count, GET /api/users/:id, PATCH /api/users/:id/role, PATCH /api/users/:id/status, DELETE /api/users/:id.

Organizaciones: GET /api/organizations/me, PATCH /api/organizations/me, DELETE /api/organizations/me, POST /api/organizations (cualquier usuario con sesión); GET /api/organizations, GET /api/organizations/:id, PATCH /api/organizations/:id/status, DELETE /api/organizations/:id (sólo Developer).

Eventos: GET /api/events/scanner-events, POST /api/events, GET /api/events/mine, GET /api/events/mine/stats (capacidad/emittidas/vendidas/ingresadas de todos los eventos vigentes del organizador en una sola tanda de consultas, fuente de la grilla "Estado de mis eventos" del Dashboard), GET /api/events/:id, PATCH /api/events/:id, PUT /api/events/:id/schedule, PUT /api/events/:id/links, GET /api/events/:id/functions/stats (mismo cálculo, por función, para un evento puntual), DELETE /api/events/:id.

Scanners de un evento (dentro de /api/events): GET /:id/scanners, POST /:id/scanners, PATCH /:id/scanners/:scannerId, POST /:id/scanners/:scannerId/disable, POST /:id/scanners/:scannerId/reactivate, POST /:id/scanners/:scannerId/revoke, POST /:id/scanners/:scannerId/regenerate, DELETE /:id/scanners/:scannerId.

Administración de entradas de un evento (dentro de /api/events): POST /:id/tickets/bulk-action, POST /:id/tickets/:ticketId/cancel, POST /:id/tickets/:ticketId/rehabilitate, POST /:id/tickets/:ticketId/reactivate, POST /:id/tickets/:ticketId/mark-used, DELETE /:id/tickets/:ticketId.

Conversación (creación de eventos por chat): POST /api/conversations/start, GET /api/conversations/:id/status, GET /api/conversations/:id, POST /api/conversations/:id/reply, DELETE /api/conversations/:id.

Ventas: GET /api/sales/mine, GET /api/sales (sólo Organizer), POST /api/sales/:id/confirm (sólo Organizer), POST /api/sales/:id/cancel, POST /api/sales/:id/resent-confirmation-email (Developer u Organizer).

Cortesías (sólo Organizer/Developer): POST /api/courtesies (emitir), GET /api/courtesies (historial con filtros), GET /api/courtesies/stats, POST /api/courtesies/:saleId/resent-email, GET /api/courtesies/:saleId/pdf, POST /api/courtesies/:saleId/cancel.

Entradas (comprador): GET /api/tickets/mine, GET /api/tickets/number/:ticketNumber, GET /api/tickets/:id/qrcode, GET /api/tickets/:id.

Entradas (organizador, listado global): GET /api/tickets/organizer.

Medios: POST /api/media/upload, DELETE /api/media/*publicId.

Módulo Scanner (sesión propia, no Clerk): GET /api/scanner/dashboard, GET /api/scanner/events, GET /api/scanner/events/:eventId/functions/:functionId/stats, GET /api/scanner/scan-attempts, POST /api/scanner/validate.

11. FUNCIONALIDADES TERMINADAS

- Autenticación de usuarios con cuenta vía Clerk, con sincronización a base propia.
- Alta, edición, aprobación y suspensión de organizaciones.
- Creación y edición de eventos por dos caminos (chat y wizard), con reglas de publicación.
- Catálogo de tipos de entrada con overrides por función.
- Checkout de compra sin cuenta, de punta a punta (sin pago real).
- Generación de entradas individuales con QR cifrado.
- Envío de email de confirmación con QR y PDF adjuntos, con reenvío.
- Recuperación de compra sin cuenta, con segundo factor por código.
- Registro único de scanner por invitación + acceso recurrente por Portal Scanner.
- Validación de QR en puerta, con garantía de consistencia bajo concurrencia real verificada con pruebas.
- Administración completa de entradas (cancelar/rehabilitar/reactivar/marcar usada/eliminar, individual y masivo) con auditoría completa.
- Panel de Developer para gestión de organizaciones y usuarios.
- Subida de imágenes (Cloudinary).
- Módulo de Cortesías (emisión, historial, estadísticas, cancelación), reutilizando de punta a punta el mismo flujo de venta/ticket/QR/email/PDF que una venta real.
- Dashboard del organizador con estadísticas reales — Comercial/Emisión/Accesos/Ocupación, una única fuente de verdad backend (`functionCapacity.service.js`) compartida entre el Dashboard y la pantalla de Entradas, con capacidad/vendidas/emitidas/ingresadas/canceladas y desglose por canal de emisión (venta/cortesía, preparado para futuros canales sin rediseño).

12. FUNCIONALIDADES PARCIALES

- **Pantalla "Tipos de entrada"**: todo real salvo la columna "Vendidas", hardcodeada en 0.
- **Administración de entradas (pantalla)**: las acciones individuales y masivas funcionan de punta a punta; la acción masiva "Exportar" es un botón que sólo muestra un aviso, sin generar ningún archivo; los filtros por estado individuales quedaron sin control visible en la pantalla actual (el estado y la llamada al backend siguen existiendo, pero no hay ningún botón/pill que lo dispare).
- **Compra**: el checkout completo funciona, pero "confirmar" no pasa por ningún pago real — es una confirmación automática simulada.
- **Cortesías**: un `DEVELOPER` no puede cancelar una cortesía de una organización que no administra (limitación heredada y deliberada, no ampliada); el historial no tiene todavía un filtro visible por canal de emisión (la estructura de datos ya lo permite sin cambios de arquitectura).

13. FUNCIONALIDADES PENDIENTES (por prioridad real, según lo que el código ya deja ver)

1. **Integración de Mercado Pago** — es la ausencia más grande del proyecto: sin esto, ninguna venta cobra dinero de verdad. Todo el resto del sistema (entradas, QR, scanner, emails, recuperación) ya está construido asumiendo que este paso va a existir.
2. **Conectar el Dashboard de Developer a datos reales** — hoy es 100% maqueta.
3. **Exportación real de entradas seleccionadas** (el botón ya existe en la UI, sólo falta la lógica).
4. **Reponer un control visible de filtro por estado** en la pantalla de administración de entradas (el backend y el estado ya lo soportan).
5. **Onboarding de datos de cobro del organizador** (ligado directamente a Mercado Pago).
6. **Notificaciones por email adicionales** (aprobación/rechazo de organización, venta nueva).
7. ~~Adaptador de WhatsApp para el motor conversacional de creación de eventos~~ — **[Actualizado 2026-08-13]** **implementado** entre el 2026-08-09 y el 2026-08-12 (posterior a la redacción original de este punto). Quedan pendientes puntuales dentro de ese adaptador, no la ausencia del adaptador en sí — ver "Actualización al 13 de agosto de 2026", sección de hallazgos UX.
8. **Páginas legales reales** (Términos y Privacidad hoy enlazan a la home).
9. **Pantalla de perfil del usuario** (hoy es un stub).
10. **Filtro visible por canal de emisión** en el historial de Cortesías (dato ya disponible, falta sólo la UI).

14. DEUDA TÉCNICA

Problemas conocidos / hallazgos concretos de código:

- En la pantalla de administración de entradas del organizador, el estado `statusFilter` y su lógica de filtrado siguen presentes y viajan al backend, pero no hay ningún control en la interfaz que permita cambiarlo — quedó código vivo sin punto de entrada visible tras una evolución posterior de la pantalla (que agregó selector de evento/función y acciones masivas).
- La acción masiva "Exportar seleccionadas" está en la interfaz pero no hace nada real (sólo un aviso).
- El campo "Vendidas" de la pantalla "Tipos de entrada" es un valor fijo, no calculado.
- El Dashboard de Developer no tiene ninguna conexión a la API — es enteramente una maqueta visual con datos de ejemplo.
- **[Resuelto]** El Dashboard del organizador y la pantalla de Entradas tenían cálculos de capacidad/vendidas/ocupación duplicados e independientes entre frontend y backend (y uno de ellos no distinguía cortesías de ventas reales, inflando "Entradas vendidas"). Se consolidaron en una única fuente backend (`functionCapacity.service.js`) consumida por ambas pantallas — el `OrganizerDataContext` ya no expone una lista completa de tickets del organizador para tallar a mano en el cliente, expone el resumen ya agregado (`eventsStats` , vía `GET /api/events/mine/stats`).
- **[Resuelto]** Las fechas de calendario (sin hora) del wizard/chat de creación de eventos podían desincronizarse un día del real en timezones detrás de UTC (Argentina incluida) — `new Date("YYYY-MM-DD")` se interpreta como medianoche UTC, no local. Afectaba al selector de fecha del wizard, a la tarjeta de revisión final del evento y, más seriamente, a la hora real guardada para eventos creados por el chat (dependía de la timezone del servidor, no de Argentina). Corregido con una única infraestructura de fechas de calendario (`lib/dateGrid.js` en el frontend, `utils/calendarDate.js` en el backend) y un offset explícito `-03:00` para toda combinación fecha+hora.

Limitaciones conocidas (documentadas en el propio código, no ocultas):

- El rate limiter es en memoria: no sobrevive un reinicio del proceso ni se comparte entre instancias si el backend llegara a correr en más de un proceso.
- Varios campos de "quién hizo esto" (el scanner de un check-in, el actor de una auditoría) se guardan como texto suelto, sin relación formal en la base de datos — funcionan bien hoy, pero no tienen integridad referencial a nivel de base.
- El canal WhatsApp del motor conversacional está modelado en la base pero no tiene ningún adaptador real.
- La timezone oficial de la plataforma (`-03:00` , Argentina) está hardcodeada como constante en dos lugares independientes (frontend y backend, son dos proyectos npm separados sin paquete común) — correcto mientras la plataforma opere sólo en Argentina; el día que soporte más de una timezone, ambos puntos hay que tocarlos.

Código duplicado identificado: normalización de email/DNI y los pequeños formateadores de fecha (los que trabajan con timestamps reales, no con fechas de calendario — ver arriba) están repetidos en varios archivos del frontend en lugar de centralizados en un único lugar (patrón usado deliberadamente en este proyecto para no crear una dependencia cruzada entre pantallas que evolucionan por separado — no necesariamente un problema a resolver, pero es duplicación real si se lo mide así). Del lado del backend, la lógica de "cuál es la organización de este usuario" (`findFirst` por `ownerId`) está repetida como una función local pequeña en más de un archivo de servicio en lugar de una única función compartida.

Módulos candidatos a mejora: el Dashboard de Developer (reconstruir sobre datos reales), la pantalla de administración de entradas (terminar exportación y reponer el filtro por estado).

15. PRÓXIMOS PASOS (orden de prioridad real, según el estado actual del código, sin inventar funcionalidades)

1. Integrar Mercado Pago (SDK, checkout, webhook, conciliación de estados) — es el único paso que falta para que el dinero se mueva de verdad.
2. Terminar la exportación de entradas seleccionadas.
3. Reponer el control de filtro por estado en la pantalla de administración de entradas.
4. Conectar el Dashboard de Developer a datos reales.
5. Calcular "Vendidas" de verdad en la pantalla de Tipos de entrada.
6. Emails adicionales (aprobación de organización, venta nueva).

7. Páginas legales reales y pantalla de perfil.

8. ~~Adaptador de WhatsApp para el motor conversacional~~ — **implementado 2026-08-09/12**; los próximos pasos reales de este frente ahora son otros (guardrail de tests, corrección de hallazgos UX puntuales, timeout de conversación) — ver la lista de próximos pasos en "Actualización al 13 de agosto de 2026", que reemplaza la prioridad de este punto.

9. Filtro visible por canal de emisión en el historial de Cortesías.

16. EVALUACIÓN GENERAL

Avance aproximado del proyecto: ~75-80%. El core funcional completo (catálogo de eventos, checkout, entradas con QR cifrado, control de acceso con garantías de concurrencia verificadas, recuperación de compra, administración de entradas con auditoría, cortesías, estadísticas reales del organizador, paneles de Developer y Organizador para todo lo que no es venta) está implementado y, en la enorme mayoría de los casos, terminado de punta a punta backend+frontend. El hueco más grande, con diferencia, sigue siendo el cobro real.

Para una versión Beta (uso real con usuarios externos pero controlado): falta, como mínimo, integrar Mercado Pago (sin esto no hay negocio real posible salvo ventas 100% manuales/gratuitas). El Dashboard del organizador ya no es un bloqueante — un organizador puede ver recaudación, ocupación y accesos reales desde el panel. El resto del sistema ya está en condiciones de sostener una Beta acotada con pagos manuales/en efectivo mientras se construye la integración de pago.

Para una versión Comercial: además de lo anterior, se necesita el Dashboard de Developer conectado a datos reales (hoy no sirve para operar el negocio), la exportación de entradas terminada, notificaciones por email adicionales (aprobación de organización, venta nueva), páginas legales reales, y revisar/formalizar las relaciones de datos que hoy son texto suelto (scanner que hizo un check-in, actor de una auditoría) si el volumen de uso lo justifica.

[Actualizado 2026-08-13] Desde la redacción original de esta evaluación (2026-08-08) se sumó un canal de creación de eventos completo por WhatsApp (bot conversacional para organizadores, ver actualización al final de este documento) — no cambia el ~75-80% de avance general (es un canal alternativo sobre funcionalidad ya contada, no una nueva pieza del ciclo de venta), pero sí introduce hallazgos de UX propios pendientes de corrección y motivó, esta misma semana, un guardrail nuevo de seguridad para tests con base de datos tras un incidente real de datos de prueba escritos en producción — ambos detallados en la actualización.

ACTUALIZACIÓN AL 13 DE AGOSTO DE 2026

Todo lo que sigue es nuevo respecto de la redacción original (2026-08-08) y está verificado contra el código y el historial Git presentes en el repositorio en este momento, no contra documentos previos. Donde algo no pudo verificarse desde este entorno, se dice explícitamente en vez de asumirse.

A1. Estado actual del proyecto

- **Branch actual:** `claude/whatsapp-db-roundtrip-optimization`.
- **Último commit incorporado (local):** `85959cb` — `test: block database tests from production`.
- **Relación con `origin/main`:** 1 commit local por delante, 0 por detrás (`git rev-list --left-right --count origin/main...HEAD` → 0 1). `origin/main` está en `3363790` (`perf(whatsapp): optimize preview publish latency`).
- **Working tree:** limpio — `git status` no muestra cambios sin commitear (el guardrail de tests con DB, tratado en A4, ya está commiteado en este branch, sólo falta el push).
- **Qué está en `origin/main` (y por lo tanto potencialmente desplegado, si Render está configurado para desplegar desde `main`):** todo el subsistema WhatsApp descrito en A2 (commits del 2026-08-09 al 2026-08-12), incluida la optimización de latencia de `PREVIEW_PUBLISH` (`3363790`).
- **Qué existe SÓLO en local, sin pushear todavía:** el guardrail de tests con base de datos (commit `85959cb`, tratado en A4).
- **Qué no puede confirmarse desde este entorno:** el estado real desplegado en Render. Este entorno no tiene acceso al panel/API de Render — no hay forma de verificar si el despliegue activo corresponde efectivamente a `origin/main` en `3363790` o a una revisión anterior, ni si existe auto-deploy configurado.
Requiere confirmación del usuario.

A2. WhatsApp para organizadores — estado real, verificado en código

El informe original (2026-08-08) describía este canal como "declarado en el modelo de datos pero sin ningún adaptador implementado". Eso dejó de ser cierto entre el 2026-08-09 y el 2026-08-12 (17 commits, ver `git log` en el repositorio). Estado punto por punto, verificado leyendo el código actual (`backend/src/controllers/whatsapp.controller.js`, `backend/src/services/whatsapp*.js`, `backend/src/conversation/EventCreationEngine.js`):

Punto	Estado	Evidencia / matiz
Webhook de Meta	Implementado	<code>GET/POST /api/whatsapp/webhook</code> , montado público en <code>app.js</code> ; verificación de token propia del handshake de Meta.
Parseo de mensajes	Implementado	<code>whatsapp.service.js</code> normaliza texto/imagen/ubicación en una forma común antes de llegar al controller.
Envío de respuestas	Implementado	<code>sendWhatsAppTextMessage</code> vía Graph API, timeout propio de 10s (<code>GRAPH_API_TIMEOUT_MS</code>), distingue <code>TIMEOUT</code> de <code>NETWORK_ERROR</code> , nunca lanza.
Identificación del responsable por teléfono	Implementado	<code>discoverWhatsappOrganizationCandidates</code> resuelve por <code>Organization.phone</code> normalizado (Argentina).
<code>WhatsappOrganizerLink</code>	Implementado	Modelo real, <code>organizationId @unique</code> (una Organization, a lo sumo un link), <code>waId</code> deliberadamente NO único (un teléfono puede administrar más de una Organization).
Descubrimiento de organizaciones aprobadas	Implementado, con un bug histórico ya corregido	Corregido en el commit <code>a261f4e</code> (2026-08-12) — ver A3.
Selector para responsables con varias organizaciones	Implementado	Siempre pregunta "¿Con cuál de tus organizaciones querés trabajar?" y lista todas — verificado en <code>buildOrganizationSelectorText</code> .
Persistencia de la organización elegida	Implementado	<code>ConversationState.organizationId</code> , resuelto una única vez al iniciar la conversación, nunca vuelto a resolver a mitad de camino.
Integración con <code>EventCreationEngine</code>	Implementado	Mismo motor que usa la Web (<code>start / resume / handleInput</code>), sin fork de lógica de negocio — WhatsApp sólo interpreta texto libre antes de llamarlo.
Creación / borrador / confirmación / publicación	Implementado	Mismo <code>EventServicePort.commit</code> que la Web.
Geocodificación	Implementado, con matiz	Ubicación compartida por WhatsApp llega con coordenadas reales; dirección manual (paso a paso) NO se geocodifica en el momento de tipearla — el geocoding server-side (<code>geocoding.service.js</code>) corre después, al confirmar el evento, compartido con la Web. Ningún caso queda sin geocodificar.
Manejo de imágenes	Parcialmente implementado	El caso feliz (llega una imagen real en el paso que la espera) funciona de punta a punta (Cloudinary). El caso "se espera una imagen y llega texto" no corrompe el estado ni avanza el paso (verificado: cae al parser del motor, que rechaza y no persiste nada), pero el mensaje de error que se le muestra al organizador es el genérico pensado para la Web ("Subí la imagen a <code>/api/media/upload</code> y mandame la URL que te devuelve"), no un mensaje de WhatsApp pidiendo la foto — ver hallazgo UX #1 en A5.
Timeout de conversación por inactividad	No implementado	No existe ningún campo de expiración en <code>ConversationState</code> ni ningún job/cron en el proyecto (confirmado: el proyecto no tiene ningún sistema de tareas programadas, ver sección 4 original) que pudiera aplicar un timeout de 15 minutos. Una conversación abandonada queda <code>ACTIVE</code> indefinidamente hasta que el organizador la retoma o la cancela explícitamente.
Reanudación	Implementado	<code>resume()</code> recupera el estado activo por <code>channelRef</code> (el número de WhatsApp) en cualquier mensaje posterior.
Idempotencia ante reintentos de Meta	No verificado — sin evidencia de protección explícita	No se encontró ningún mecanismo de deduplicación por <code>messageId / wamid</code> en <code>whatsapp.controller.js</code> . Meta puede reentregar el mismo webhook más de una vez; no está probado qué pasa si eso ocurre mientras el mensaje ya fue procesado (en el peor caso, podría interpretarse como un segundo mensaje idéntico del usuario). No confirmado como bug real — sólo no hay evidencia de que esté protegido.

Punto	Estado	Evidencia / matiz
Concurrencia (dos mensajes casi simultáneos del mismo número)	No verificado	No se encontró ningún <code>\$transaction /lock</code> explícito alrededor de la lectura+escritura de <code>ConversationState</code> por mensaje, a diferencia del módulo Scanner (que sí tiene esta garantía verificada con pruebas de concurrencia real, ver sección 7 original). Distinto del caso Scanner porque un mismo organizador difícilmente mande dos mensajes en el mismo milisegundo, pero queda como un supuesto no probado, no como una garantía verificada.
Instrumentación de latencia	Implementado	<code>whatsappPerf.js</code> : timer por mensaje (<code>[WA_PERF]</code>), gateado por <code>WHATSAPP_PERF_LOG</code>), conteo y duración de cada query Prisma real vía <code>instrumentPrismaClient</code> (Prisma Client Extensions + <code>AsyncLocalStorage</code>), y medición de llamadas externas (<code>timeExternalCall</code> , usado para el geocoding). Nunca loguea PII ni datos de negocio, sólo nombres de operación y duraciones.
Optimización de duplicados de <code>ConversationState</code>	[Actualizado 2026-08-14] Implementada	Persistida en <code>0ea5ef2</code> y <code>47324ee</code> — ver B1. El camino normal de <code>HANDLE_INPUT</code> pasó de 3 a 2 consultas.

A3. Últimos incidentes y correcciones (orden cronológico)

1) Bug de descubrimiento incompleto — "aparece Cine Nadia pero no La Taberna de Mou". Causa comprobada en código: la versión anterior de `discoverWhatsappOrganizationCandidates` hacía `return` apenas encontraba **cualquier** `WhatsappOrganizerLink` ya existente para el número, sin volver a consultar `Organization.phone` — así que una segunda `Organization` con el mismo teléfono, pero sin link todavía creado, nunca se descubría. No era un problema de unicidad de ningún campo del schema (`waId` nunca fue único). **Corregido** en el commit `a261f4e` (2026-08-12, `fix(whatsapp): resolve all organizations linked to phone`): ahora se consultan siempre las dos fuentes (links existentes + `Organizations` aprobadas sin link cuyo teléfono coincide) y se unen deduplicando exclusivamente por `organizationId`. Estado: **resuelto y verificado en código** (test de regresión específico en `tests/whatsappOrganizerDiscovery.test.js`).

2) Datos de prueba escritos en producción — organizaciones duplicadas en el selector. El mismo fix de `a261f4e`, al dejar de esconder resultados, expuso un problema de datos preexistente: 52 filas `Organization` (36 `ownerId` distintos, ninguno con eventos reales) aparecieron en producción con nombres "Cine Nadia" / "La Taberna de Mou" / "Organización Pendiente", en ráfagas de milisegundos/segundos el 2026-08-12 (~23:03, ~23:04, ~23:17). Causa comprobada, no supuesta: el archivo `backend/tests/whatsappOrganizerDiscovery.test.js` crea 13 `Organizations` y 9 `Users` reales por corrida completa (9 "Cine Nadia" + 3 "La Taberna de Mou" + 1 "Organización Pendiente" — coincide exactamente, dígito por dígito, con la primera ráfaga observada; $13 \times 4 = 52$ y $9 \times 4 = 36$ coinciden exactamente con el total, reconstruyendo 4 corridas). El guard de ese archivo (`Boolean(process.env.DATABASE_URL)`) sólo confirmaba que la variable existiera, nunca a qué proyecto apuntaba — si la terminal donde se corrió `node --test` tenía un `backend/.env` real cargado (no existe `.env.test` en ningún punto anterior del proyecto), los tests corrían de verdad contra producción. El `finally/cleanup` de cada test existe y está bien escrito, pero no llegó a ejecutarse en las 4 corridas — mecanismo exacto no confirmable sin logs externos a este entorno; hipótesis mejor fundada: el proceso se interrumpió (Ctrl+C o timeout) antes de terminar, plausible dado que cada corrida completa contra producción podía tardar 60-100+ segundos por la latencia real de ~900ms/query medida en esas fechas. **Estado: causa identificada con certeza (coincidencia numérica exacta), mecanismo de interrupción no confirmado al 100%, guardrail de código ya implementado (ver A4) — limpieza de los datos duplicados en producción NO realizada todavía, sigue pendiente y fuera del alcance de este informe (es una tarea de escritura en producción, no de diagnóstico).**

3) Tercer archivo con acceso real a Prisma — `eventServicePort.commit.perf.test.js`. Al auditar el repositorio completo para el guardrail (no sólo los dos archivos ya conocidos), apareció un tercer archivo con el mismo patrón débil de guard, incorporado por el commit `3363790` (`perf(whatsapp): optimize preview publish latency`, ya en `origin/main`, no escrito en esta sesión). No hay evidencia de que este archivo haya contribuido al incidente de datos duplicados (su alcance es `Event/EventLink`, nunca `Organization/WhatsappOrganizerLink` — confirmado leyendo el diff completo del commit), pero tenía el mismo riesgo estructural. **Estado: identificado y corregido en el mismo guardrail (ver A4).**

4) Separación tests unitarios / tests con DB, y suite completa contra base de test real. Se agregaron 3 scripts (`npm test`, `npm run test:unit`, `npm run test:db`, ver A4). **Estado: los scripts existen y funcionan; la ejecución real contra una base de test todavía NO fue posible desde ningún entorno de esta sesión**

— no existe un `backend/.env.test` real con credenciales, sólo la plantilla `backend/.env.test.example`. Sigue pendiente, ver A7 y A9.

A4. Guardrail de producción para tests con DB

Implementado el 2026-08-13, commiteado (85959cb , este mismo branch), **no pusheado todavía**.

- `backend/tests/helpers/dbGuard.js` — módulo centralizado. Lee `DATABASE_URL` y `DIRECT_URL`. Si cualquiera contiene el project-ref de producción `oiyakkbvplxrysjsxwrf`, lanza una excepción inmediatamente al importarse (antes de que el archivo que lo importa defina un solo `test(...)`, y mucho antes de poder crear un fixture) — nunca imprime la connection string. Si contiene el project-ref de test `ldxmrsllusbnayerxtbu`, exporta `hasDatabase = true`. Si está ausente o es cualquier otro proyecto, `hasDatabase = false` — nunca escribe, nunca aborta.
- `backend/tests/helpers/loadTestEnv.js` — preload vía `node --import`, carga `backend/.env.test` si existe (nunca `.env`); si el archivo no existe todavía, no falla, sólo no carga nada.
- **Scripts en `backend/package.json`**: `test` (suite completa con `.env.test` cargado), `test:unit` (rápido, sin cargar `.env.test`), `test:db` (sólo los 3 archivos que tocan Prisma real, con `.env.test` cargado).
- `backend/.env.test.example` — plantilla commiteada, sin credenciales reales, documentando los dos placeholders (`DATABASE_URL` / `DIRECT_URL`) y el project-ref esperado.
- `backend/tests/dbGuard.test.js` — 10 tests nuevos, sin ninguna base real: 5 sobre la función pura de clasificación, 5 vía subproceso Node real que importa `dbGuard.js` con variables de entorno controladas, probando los 3 casos pedidos (producción → aborta antes de cualquier operación Prisma; test → habilita; ausente/otro proyecto → nunca escribe).
- **Archivos protegidos por el guard centralizado (los 3 únicos en todo `backend/tests/` que tocan Prisma real con escritura, confirmado por barrido completo del directorio):**
`whatsappOrganizerDiscovery.test.js`, `whatsappPendingStepInput.service.test.js`, `eventServicePort.commit.perf.test.js`.
- **Resultados comprobados ahora mismo, en este entorno (re-verificado, no copiado de un informe anterior):** `npm run test:unit` → 521 tests, 489 pass, 0 fail, 32 skipped. `npm run test:db` → 45 tests, 13 pass, 0 fail, 32 skipped (los 32 son exactamente los `testWithDb` de los 3 archivos protegidos, saltados porque no hay `backend/.env.test` en este entorno).
- **Tests omitidos por falta de una DB de test configurada:** los 32 anteriores — nunca se ejecutaron contra ninguna base real desde este entorno.
- **Riesgos residuales:** (1) el guard es una comparación de substring sobre el project-ref, no una validación criptográfica de identidad — suficiente dado cómo Supabase nombra sus hosts, pero no infalible ante un project-ref manualmente falseado en la URL; (2) no hay todavía un `backend/.env.test` real, así que el camino "tests con DB" nunca fue ejercitado de punta a punta contra Postgres real, sólo simulado con URLs falsas en subprocesos de prueba; (3) los 52 registros duplicados de producción (A3, incidente 2) siguen sin limpiar.
- **Estado de commit:** commiteado en 85959cb sobre `claude/whatsapp-db-roundtrip-optimization`, **sin push**.

A5. Hallazgos de las pruebas UX del 2026-08-13

Verificado contra el código actual, no contra lo que documentos previos afirmaban. No se tuvo acceso a logs/transcripciones reales de la sesión de pruebas del 13/08 desde este entorno — cada fila es el resultado de leer el código correspondiente, no de reproducir la sesión de pruebas.

#	Requisito	Estado	Evidencia
1	Imagen esperada, llega texto → no avanzar/corromper, pedir imagen	Parcial	No corrompe estado ni avanza el paso (confirmado). El mensaje mostrado es el error genérico de Web ("Subí la imagen a /api/media/upload...", <code>inputHandlers/imageUrl.js</code>) — <code>extractWhatsappReplyText</code> no tiene una rama para <code>IMAGE_URL</code> , así que no hay un mensaje de WhatsApp pidiendo la foto.
2	Paso de ubicación sin dirección precargada automáticamente	Implementado	<code>findReusableOrganizationLocation</code> sólo ofrece reutilizar una dirección anterior de la Organization — requiere confirmación explícita ("1"/"2"); si el organizador elige "2", el <code>pending</code> se limpia a <code>{}</code> antes de pedir la dirección nueva. El draft nunca se prellena solo.
3	No repetir "¿Vamos a publicar con {org}?"	Pendiente — bug real confirmado en código	<code>buildOrganizationSelectedConfirmationText</code> (<code>whatsappOrganizerBot.service.js</code>) arma exactamente ese texto después de que el organizador ya eligió una organización del selector

#	Requisito	Estado	Evidencia
	Sí/No" tras selección explícita		— y ya había respondido "sí" a "¿Querés publicar un evento?" en el saludo genérico previo al selector. Es una pregunta redundante real, no un falso positivo.
4	Con más de una organización, siempre preguntar "¿Con cuál querés trabajar?"	Implementado	<code>buildOrganizationSelectorText</code> , siempre se muestran todos los candidatos.
5	Incidente histórico "Cine Nadia sí, La Taberna de Mou no"	Documentado, corregido — ver A3, incidente 1. Distinto del incidente de datos duplicados (A3, incidente 2), que es posterior y de otra naturaleza.	
6	No aceptar fechas anteriores al día actual	Implementado	<code>isArgentineDateInThePast</code> , aplicado en los 3 caminos de carga de fecha (función única, lista de funciones, rango recurrente) — verificado los 3 call sites en <code>whatsapp.controller.js</code> .
7	Modalidad de funciones: una / varias / recurrentes	Implementado	<code>Step FUNCTIONS_MODE</code> , 3 opciones (<code>SINGLE</code> / <code>MULTIPLE</code> / <code>RECURRING</code>).
8	Ubicación: compartir por WhatsApp o dirección manual	Implementado	<code>tryHandleLocationSubflow</code> , dos caminos explícitos.
9	Opción "Volver" en cualquier paso	Implementado	<code>isBackCommand</code> con manejo central para pasos sin sub-flujo propio, más manejo específico de "volver" dentro de cada sub-flujo (ubicación, funciones, entradas).
10	Timeout de 15 minutos por inactividad	No implementado	Sin campo de expiración en el modelo, sin ningún job/cron en el proyecto que pudiera aplicarlo.
11	Mensajes con opciones numeradas, formato explicado	Implementado	Patrón consistente en todo el bot ("1. Sí\n2. No", listas numeradas de opciones).
12	Precio ingresado sin símbolo \$	Implementado	<code>inputHandlers/price.js</code> acepta un número simple; el bot instruye explícitamente "sin el signo \$"; el \$ sólo se reagrega al mostrar un precio ya cargado, nunca al pedirlo.

A6. Rendimiento y latencia

- **Instrumentación existente:** `whatsappPerf.js` — timer por mensaje, conteo/duración de cada query Prisma real (vía Prisma Client Extensions + `AsyncLocalStorage`, sin tocar cada service individualmente), medición de llamadas externas (geocoding). Gateada por `WHATSAPP_PERF_LOG`, nunca activa por defecto, nunca loguea PII.
- **Cuellos de botella comprobados (con evidencia real de producción, en fases anteriores a esta actualización):** la enorme mayoría de la latencia de una interacción de WhatsApp está dentro de Prisma/Postgres, no en lógica de negocio ni en Meta; incluso un `findUnique` por clave primaria mostró una latencia uniforme alta, consistente con overhead de red/pooler entre Render y Supabase más que con complejidad de la query — diagnosticado, **nunca resuelto a nivel de infraestructura** (fuera del alcance autorizado en su momento).
- **Lecturas duplicadas de `ConversationState` : [Actualizado 2026-08-14] resueltas en el camino normal.** La cache transparente (`AsyncLocalStorage`) diseñada acá se persistió en `0ea5ef2`, y `47324ee` la terminó de poblar en el punto que quedaba vacío (`findActiveConversation`) — ver B1. `HANDLE_INPUT` pasó de 3 a 2 consultas de `ConversationState`.
- **Índice propuesto sobre `ConversationState` (`channel`, `channelRef`, `status`, `createdAt`) :** sigue **propuesto, no aplicado** — ninguna sesión de este proyecto tocó el schema para esto.
- **Optimizaciones ya aplicadas y confirmadas en el código actual:** `syncEventLinksService` / `syncEventScheduleService` con parámetro `returnEvent:false` para evitar una lectura pesada descartada; `EventLink` pasó de un loop de `create` a un único `createMany`; instrumentación de la llamada externa de geocoding (antes invisible dentro de "tiempo de base de datos"). Todo esto corresponde al commit `3363790`, ya en `origin/main`.

- **Resultados antes/después:** no se incluye ningún número de milisegundos específico en esta actualización salvo los ya documentados en informes de fases anteriores de este mismo repositorio — en particular, la cifra de " `LOCATION_CONFIRMATION_PROMPT` ~5176 ms" mencionada como antecedente posible **no se encontró en ningún archivo, log o commit del repositorio** (se buscó explícitamente) y por lo tanto no se incluye como dato verificado; `LOCATION_CONFIRMATION_PROMPT` sí existe en el código como nombre de marca de instrumentación (`whatsapp.controller.js`), pero sin ningún valor de latencia asociado guardado en el repositorio.
- **Optimizaciones pendientes: [Actualizado 2026-08-14]** cache de `ConversationState` , `resetPendingStepInput` como `upsert` y paralelización de `discoverWhatsappOrganizationCandidates` — las 3 aplicadas y verificadas, ver B1. Sigue sin aplicarse, por decisión explícita (ver B4): la investigación de infraestructura (región Render/Supabase, modo de conexión del pooler).

A7. Base Supabase de TEST

- El proyecto de test **ya fue creado** (dato provisto por el usuario, no verificable desde este entorno sin credenciales).
- Project-ref esperado: `ldxmrsllusbnayerxtbu` .
- La base de **producción** (`oiyakkbvplxrysjsxwrf`) no debe borrarse ni modificarse — el guardrail de A4 existe específicamente para esto.
- **Requiere confirmación del usuario:** si `backend/.env.test` ya existe localmente en la máquina del usuario y si sus credenciales corresponden efectivamente al project-ref de test. Desde este entorno se confirmó que `backend/.env.test` **no existe y no está trackeado en git** (correcto, así debe ser).
- No se incluyó ninguna credencial ni URL real en este informe, en ningún punto.
- Próxima validación pendiente: ejecutar `npm run test:db` con `backend/.env.test` apuntando de verdad al project-ref de test, confirmando el project-ref ANTES de correr (ver A9).

A8. Estado de pruebas

Comando	Aprobados	Fallidos	Omitidos	¿Usa DB real?	Evidencia
<code>npm run test:unit</code>	489	0	32	No	Re-ejecutado en este entorno, 2026-08-13
<code>npm run test:db</code> (sin <code>.env.test</code>)	13	0	32	No — el guard confirma <code>ABSENT</code> , los <code>testWithDb</code> se saltan	Re-ejecutado en este entorno, 2026-08-13
<code>npm run test:db</code> con <code>.env.test</code> de prueba conteniendo el project-ref de PRODUCCIÓN (credenciales falsas, sólo para probar el guard)	0	3 (las 3 falsas, esperado)	—	No — abortó antes de cualquier operación Prisma	Ejecutado y revertido en la sesión donde se implementó el guardrail (2026-08-13)
<code>npm run test:db</code> contra la base de TEST real	—	—	—	—	No ejecutado — requiere <code>backend/.env.test</code> real, no disponible en este entorno

No se ejecutó ningún test con DB contra ningún project-ref sin poder confirmar antes cuál era, tal como se pidió.

A9. Próximos pasos

1. Revisar y **pushear** el guardrail (`85959cb`), que hoy sólo existe local en `claude/whatsapp-db-roundtrip-optimization` .
2. Verificar (ya hecho, re-confirmar antes de cada corrida) que `backend/.env.test` no esté trackeado en git.
3. Configurar `backend/.env.test` localmente con las credenciales reales de la base TEST (`ldxmrsllusbnayerxtbu`) — pendiente, fuera del alcance de este entorno.
4. Comprobar explícitamente el project-ref antes de ejecutar cualquier test con DB (el guard ya lo hace automáticamente, pero conviene una verificación manual la primera vez).
5. Ejecutar `npm run test:db` contra esa base real por primera vez.

6. Revisar resultados y la limpieza de fixtures **únicamente en TEST** — nunca en producción.
7. Continuar las pruebas UX del canal WhatsApp, en particular los 2 hallazgos pendientes de A5 (mensaje de imagen no esperada, pregunta redundante de confirmación) y la ausencia de timeout por inactividad.
8. Consolidar las correcciones UX de A5 en una implementación controlada, con el mismo criterio de bajo riesgo usado en el resto de este proyecto (no mezclar con la optimización de latencia).
9. ~~Retomar la optimización de latencia (cache de ConversationState, upsert de WhatsappPendingStepInput, paralelización de discovery — todas diseñadas en A6 pero no persistidas) recién después de asegurar que la suite de tests corre de forma segura contra la base de TEST real.~~ — **[Actualizado 2026-08-14] hecho**, ver B1-B5. Fase de optimización de latencia de WhatsApp cerrada.

Pendiente aparte, no incluido en la lista anterior porque implica escribir en producción (fuera del alcance de esta actualización, que es sólo documental): limpiar los 52 registros duplicados de `Organization` identificados en A3, incidente 2 — requiere que el usuario confirme cuál de cada grupo duplicado es el registro real a conservar.

ACTUALIZACIÓN AL 14 DE AGOSTO DE 2026

Resumen consolidado de la optimización de latencia de WhatsApp diseñada en la actualización anterior (A6) y cerrada en la fecha de esta actualización. Verificado contra el historial Git y los tests ejecutados en este entorno; las cifras de latencia post-deploy fueron provistas por el usuario a partir de logs `[WA_PERF]` reales de producción — no verificables desde este entorno (sin acceso a Render). No se incluye ningún log completo, message ID, teléfono, credencial ni URL completa en esta sección.

B1. Commits de esta fase (en orden)

1. **0ea5ef2** — `perf(whatsapp): reduce database operations per message`. Consolidó en código tres optimizaciones que la actualización anterior había diseñado pero no persistido (ver A6): cache de `ConversationState` acotada a un único mensaje (`conversationStateRequestCache.js`, vía `AsyncLocalStorage`, nunca compartida entre mensajes concurrentes), `resetPendingStepInput` convertido de `deleteMany + create` a un único `upsert`, y paralelización (`Promise.all`) de las dos consultas independientes de `discoverWhatsappOrganizationCandidates`.
2. **47324ee** — `perf(whatsapp): reuse active conversation lookup`. La cache introducida por `0ea5ef2` quedaba vacía en el camino más común porque `findActiveConversation()` usaba un `select` parcial (`id / userId / organizationId`) que no alcanzaba para cachearse — `resume()` seguía pagando su propio `findUnique` completo. Se amplió la consulta a la fila completa y se pobló la cache ahí mismo (sin cambiar el contrato público de `findActiveConversation`, que sigue devolviendo sólo esos 3 campos a sus callers). El camino normal de `HANDLE_INPUT` bajó de 3 consultas (`findFirst + findUnique + updateMany`) a 2 (`findFirst + updateMany`). Sin impacto en Web: `findActiveConversation` es exclusivo del adaptador de WhatsApp.
3. **5a1c8e3** — `perf(whatsapp): reduce preview draft queries`. En la rama `DRAFT` de `EventServicePort.commit()`, la lectura final (`getMyEventByIdService`) repetía la resolución de `User / Organization` que `createEventService` ya había hecho segundos antes en la misma llamada, y además corría un self-heal de archivado que nunca puede encontrar nada en un evento recién creado (todavía en estado `DRAFT`, el filtro del self-heal sólo mira `PUBLISHED / FINISHED / CANCELLED`). Se reemplazó por una lectura directa (`getEventWithDetailsById`, nueva, sin re-chequeo de pertenencia ni self-heal) — 3 operaciones Prisma menos por cada borrador guardado, verificado con conteo exacto contra la base de TEST. **PREVIEW_PUBLISH no se tocó**: se verificó que su desglose de operaciones quedó idéntico, antes y después.
4. **336f6e0** — `fix(whatsapp): stop suggesting previous event location`. Eliminada la consulta `Event.findFirst` que, al llegar al paso de ubicación, ofrecía reutilizar la dirección de un evento anterior de la Organization, junto con el mensaje "¿el evento es en {lugar}?" asociado. El flujo pasa ahora directo al selector de método (1. Compartir ubicación / 2. Ingresar dirección manual), con el estado siempre vacío — sin dirección, coordenadas ni ningún dato precargado. El status `AWAITING_REUSE_CONFIRMATION` se dejó de generar para conversaciones nuevas, pero el sub-flujo todavía sabe procesarlo: una conversación vieja que ya estuviera esperando esa respuesta puede seguir respondiendo con normalidad, sin quedar trabada.

B2. Resultados medidos (provistos por el usuario, logs `[WA_PERF]` de producción)

No verificables desde este entorno — no hay acceso a los logs de Render. Se documentan tal como fueron reportados, sin reproducir ninguna línea de log completa:

- `HANDLE_INPUT` : mediana aproximada de ~3,24 s a ~2,62 s después de `47324ee` ; muestras posteriores en el rango 2,3-2,7 s.
- Flujo de imagen: de ~7,86 s a ~6,42 s, medido antes de la última simplificación (`336f6e0`) — no se reportó todavía una cifra posterior a ese commit.
- `RESUME` : prácticamente 0 ms en el camino normal, consistente con que `47324ee` le elimina el `findUnique` propio.
- Todas las muestras reportadas con `success:true` .

B3. Tests

Ejecutados y verificados en este entorno para cada uno de los 3 commits de código de esta fase (`47324ee` , `5a1c8e3` , `336f6e0`), en cada caso con el guardrail de A4 (`dbGuard.js`) confirmando el project-ref de TEST (`ldxmrsllusbnayerxtbu`) y la ausencia del de producción antes de correr cualquier test con DB:

- `npm run test:unit` : verde en los 3 puntos (505 pass / 0 fail en el estado final de la fase).
- `npm run test:db` : verde en los 3 puntos (66/66 pass / 0 fail en el estado final), corrido contra la base de TEST real — primera vez que este comando se ejecutó contra una base real desde que existe el guardrail (A7 lo dejaba pendiente).
- Se agregaron tests de conteo exacto de operaciones Prisma (antes/después) para `47324ee` y `5a1c8e3` , reutilizando la instrumentación `[WA_PERF]` ya existente contra datos reales de TEST, sin mocks de Prisma.
- `336f6e0` no requirió tests con DB (cambio acotado a `whatsapp.controller.js` , capa con inyección de dependencias mockeada) — cubierto con tests unitarios nuevos más la suite preexistente de ubicación (sin modificar), que ya prueba de punta a punta dirección manual, ubicación compartida y la compatibilidad con conversaciones viejas en `AWAITING_REUSE_CONFIRMATION` .

B4. Decisiones de alcance (explícitas del usuario para esta fase)

- No seguir optimizando los pasos "planos" del motor conversacional (nombre, descripción, categoría, etc.) — margen de mejora marginal frente al riesgo de seguir tocando el camino caliente.
- Los ~20 segundos de la operación de publicación (`PREVIEW_PUBLISH`) se aceptan tal cual: es una operación única por evento, no repetida por mensaje, y no una candidata prioritaria de optimización.
- No se modifica Render, Supabase, el pooler de conexión ni ningún plan contratado — la causa de fondo diagnosticada en A6 (latencia de infraestructura, no de código de aplicación) queda documentada pero fuera de alcance por decisión explícita.

B5. Cierre de esta fase

Con estos 4 commits, la optimización de latencia de WhatsApp diseñada en A6 y las lecturas duplicadas de `ConversationState` señaladas ahí mismo como pendientes quedan **implementadas y verificadas**. Se da por **cerrada** la fase de optimización de latencia de WhatsApp. Los puntos que siguen abiertos (B6) son de otra naturaleza — validación funcional/UX, no latencia — y quedan para una fase posterior, no como continuación directa de ésta.

B6. Pendientes (fuera del alcance de esta fase)

- Respuesta cuando llega un video en el paso que espera una imagen (variante puntual de A5 #1, no reverificada en esta fase).
- Validación de fechas anteriores al día actual como parte de una prueba final integral — A5 #6 ya documenta esto como implementado en código (`isArgentineDateInThePast`); lo que queda pendiente es la re-validación funcional, no una corrección de código conocida.
- Verificación funcional de los 3 modos de función (única/múltiples/recurrentes) como parte de la misma prueba integral — A5 #7 ya lo documenta como implementado en código; misma salvedad que el punto anterior.
- Prueba final integral del canal WhatsApp de punta a punta (no ejecutada en este entorno).

- Integración de Mercado Pago — confirmado que sigue para una fase posterior, sin cambios respecto de las secciones 13/15 del informe original.

B7. Puntos de secciones previas superados por esta fase

Los siguientes puntos de la actualización del 13/08 quedan reemplazados por B1-B5 (se dejan anotados en su lugar original en vez de reescribirse, siguiendo el mismo criterio que ya usa este informe en las secciones 13 y 15): la fila "Optimización de duplicados de `ConversationState`" de A2 (era "Diseñada, NO implementada"), el bloque de "Lecturas duplicadas de `ConversationState`" y "Optimizaciones pendientes" de A6 (`upsert` de pending y paralelización de discovery incluidos), y el punto 9 de A9 — los tres pasan a **implementados**, ver B1-B3.